```
SET SERVEROUTPUT ON;
SET SERVEROUTPUT ON SIZE UNLIMITED;


/* ============================================================
   CHAPTER 3: SQL WITHIN PL/SQL
   Topics:
   - SELECT INTO
   - INSERT
   - UPDATE
   - DELETE
   - MERGE
   - COMMIT
   - ROLLBACK
   - SAVEPOINT
   - %TYPE
   - %ROWTYPE
   Labs:
   - Employee Management System
   - Customer Update Program
   - Transaction Rollback Example
   ============================================================ */

SET SERVEROUTPUT ON SIZE UNLIMITED;

/* ============================================================
   1. DROP OLD TABLES
   ============================================================ */

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE employees CASCADE CONSTRAINTS';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE customers CASCADE CONSTRAINTS';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'DROP TABLE employee_backup CASCADE CONSTRAINTS';
EXCEPTION
    WHEN OTHERS THEN NULL;
END;
/
```

```
/* ============================================================
   2. CREATE TABLES
   ============================================================ */

CREATE TABLE employees
(
    employee_id   NUMBER PRIMARY KEY,
    employee_name VARCHAR2(100),
    department    VARCHAR2(50),
    salary        NUMBER,
    status        VARCHAR2(20)
);

CREATE TABLE customers
(
    customer_id   NUMBER PRIMARY KEY,
    customer_name VARCHAR2(100),
    phone         VARCHAR2(30),
    city          VARCHAR2(50),
    status        VARCHAR2(20)
);

CREATE TABLE employee_backup
(
    employee_id   NUMBER PRIMARY KEY,
    employee_name VARCHAR2(100),
    department    VARCHAR2(50),
    salary        NUMBER,
    status        VARCHAR2(20)
);

/* ============================================================
   3. INSERT SAMPLE DATA
   ============================================================ */

INSERT INTO employees VALUES (1, 'Aung Aung', 'IT', 800000, 'ACTIVE');
INSERT INTO employees VALUES (2, 'Mg Mg', 'HR', 600000, 'ACTIVE');
INSERT INTO employees VALUES (3, 'Su Su', 'Finance', 900000, 'ACTIVE');

INSERT INTO customers VALUES (1, 'Myanmar Telecom Co., Ltd', '091111111',
'Yangon', 'ACTIVE');
INSERT INTO customers VALUES (2, 'Global Trading Co., Ltd', '092222222',
'Mandalay', 'ACTIVE');

COMMIT;
```

```
/* ============================================================
   4. SELECT INTO Example
   ============================================================ */
```

```
SQL> DECLARE
    v_name   employees.employee_name%TYPE;
    v_salary employees.salary%TYPE;
BEGIN
    SELECT employee_name, salary
    INTO v_name, v_salary
    FROM employees
    WHERE employee_id = 1;

    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13
Employee Name: Aung Aung
Salary: 800000

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   5. INSERT Statement inside PL/SQL
   ============================================================ */
```

```
SQL> BEGIN
    INSERT INTO employees
    (
        employee_id,
        employee_name,
        department,
        salary,
        status
    )
    VALUES
    (
        4,
        'Hla Hla',
        'Sales',
        500000,
        'ACTIVE'
    );

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('New employee inserted success
fully');
END;
/
  2    3    4    5    6    7    8    9   10   11   12
 13   14   15   16   17   18   19   20   21   22   23
New employee inserted successfully

PL/SQL procedure successfully completed.
```

```
SQL> SELECT * FROM employees;

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                          SALARY
--------------------------------------------------- ----------
STATUS
--------------------
          1
Aung Aung
IT                                                  800000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                          SALARY
--------------------------------------------------- ----------
STATUS
--------------------
          2
Mg Mg
HR                                                  600000
ACTIVE
EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                          SALARY
--------------------------------------------------- ----------
STATUS
--------------------
          3
Su Su
Finance                                             900000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                          SALARY
--------------------------------------------------- ----------
STATUS
--------------------
          4
Hla Hla
Sales                                               500000
ACTIVE
```

```
/* ============================================================
   6. UPDATE Statement inside PL/SQL
   ============================================================ */
```

```
SQL> BEGIN
    UPDATE employees
    SET salary = salary + 100000
    WHERE employee_id = 4;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee salary updated successfully');
END;
/
  2    3    4    5    6    7    8    9   10  Employee salary updated successfully

PL/SQL procedure successfully completed.
```

```
SQL> SELECT * FROM employees
WHERE employee_id = 4;
  2
EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------
DEPARTMENT                                            SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         4
Hla Hla
Sales                                                 600000
ACTIVE
```

```
/* ============================================================
   7. DELETE Statement inside PL/SQL
   ============================================================ */
```

```
SQL> BEGIN
    DELETE FROM employees
    WHERE employee_id = 4;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee deleted successfully');
END;
/
  2    3    4    5    6    7    8    9  Employee deleted successfully

PL/SQL procedure successfully completed.
```

```
SQL> SELECT * FROM employees
WHERE employee_id = 4;
  2
no rows selected
```

```
/* ============================================================
   8. MERGE Statement inside PL/SQL
   Purpose:
   - If employee exists in backup table, update it
   - If employee does not exist, insert it
   ============================================================ */
```

```
SQL> BEGIN
    MERGE INTO employee_backup b
    USING employees e
    ON (b.employee_id = e.employee_id)
    WHEN MATCHED THEN
        UPDATE SET
            b.employee_name = e.employee_name,
            b.department    = e.department,
            b  2  .salary        = e.salary,
            b.status        = e.status
    WHEN NOT MATCHED THEN
        INSERT
        (
            employee_id,
            employee_name,
            department,
            salary,
            status
        )  3
        VALUES
        (
            e.employee_id,
            e.employee_name,
            e.department,
            e.sa  4  lary,
            e.status
        );

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee backup table merged successfully');
END;
/
  5    6    7    8    9   10   11   12   13   14   15   16   17   18   1
9   20   21   22   23   24   25   26   27   28   29   30   31   32   33
 Employee backup table merged successfully

PL/SQL procedure successfully completed.
```

```
SQL> SELECT * FROM employees;

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------
DEPARTMENT                                          SALARY
--------------------------------------------------- ----------
STATUS
--------------------
         1
Aung Aung
IT                                                  800000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------
DEPARTMENT                                          SALARY
--------------------------------------------------- ----------
STATUS
--------------------
         2
Mg Mg
HR                                                  600000
ACTIVE
```

```
SQL> SELECT * FROM employee_backup;

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------
DEPARTMENT                                          SALARY
--------------------------------------------------- ----------
STATUS
--------------------
         1
Aung Aung
IT                                                  800000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------
DEPARTMENT                                          SALARY
--------------------------------------------------- ----------
STATUS
--------------------
         2
Mg Mg
HR                                                  600000
ACTIVE
```

```
/* ============================================================
   9. Transaction Control: COMMIT
   ============================================================ */
```

```
SQL> BEGIN
    INSERT INTO employees
    VALUES (5, 'Kyaw Kyaw', 'Admin', 450000, 'ACTIVE');

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Transaction saved permanently using COMMIT');
END;
/
  2    3    4    5    6    7    8    9  Transaction saved permanently using COMMIT

PL/SQL procedure successfully completed.
```

```
SELECT * FROM employees;
```

```
EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                                  SALARY
---------------------------------------------- ----------
STATUS
--------------------
         5
Kyaw Kyaw
Admin                                                       450000
ACTIVE
```

```
/* ============================================================
   10. Transaction Control: ROLLBACK
   ============================================================ */
```

```
SQL> BEGIN
    INSERT INTO employees
    VALUES (6, 'Temporary User', 'Test', 300000, 'ACTIVE');

    ROLLBACK;

    DBMS_OUTPUT.PUT_LINE('Transaction cancelled using ROLLBACK');
END;
/
  2    3    4    5    6    7    8    9  Transaction cancelled using ROLLBACK

PL/SQL procedure successfully completed.
```

```
SELECT * FROM employees;
```

```
/* ============================================================
   11. Transaction Control: SAVEPOINT
   ============================================================ */
```

```
SQL> BEGIN
    INSERT INTO employees
    VALUES (7, 'Savepoint User 1', 'IT', 400000, 'ACTIVE');

    SAVEPOINT sp_after_first_insert;

    INSERT INTO employees
    VALUES (8, 'Savepoint User 2', 'IT', 420000, 'ACTIVE');

    ROLLBACK TO sp_after_first_insert;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Only employee 7 is saved. Employee 8 is rolled
back.');
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14   15   16
  Only employee 7 is saved. Employee 8 is rolled back.

PL/SQL procedure successfully completed.
```

```
SELECT * FROM employees;
```

```
EMPLOYEE_ID
-----------
EMPLOYEE_NAME
------------------------------------------------------------
DEPARTMENT                                           SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         7
Savepoint User 1
IT                                                   400000
ACTIVE
```

```
/* ============================================================
   12. Working with %TYPE
   Purpose:
   - Use column data type automatically
   ============================================================ */
```

```
SQL> DECLARE
    v_employee_name employees.employee_name%TYPE;
    v_department    employees.department%TYPE;
BEGIN
    SELECT employee_name, department
    INTO v_employee_name, v_department
    FROM employees
    WHERE employee_id = 1;

    DBMS_OUTPUT.PUT_LINE('Name: ' || v_employee_name);
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_department);
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13
Name: Aung Aung
Department: IT

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   13. Working with %ROWTYPE
   Purpose:
   - Store one full table row into a variable
   ============================================================ */
```

```
SQL> DECLARE
    v_employee employees%ROWTYPE;
BEGIN
    SELECT *
    INTO v_employee
    FROM employees
    WHERE employee_id = 2;

    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_employee.employee_id);
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_employee.employee_name
);
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_employee.department);
    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_employee.salary);
    DBMS_OUTPUT.PUT_LINE('Status: ' || v_employee.status);
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14   15
Employee ID: 2
Employee Name: Mg Mg
Department: HR
Salary: 600000
Status: ACTIVE

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   LAB 1: Employee Management System
   Requirement:
   - Insert new employee
   - Update salary based on department
   - Display employee information
   ============================================================ */
```

```
SQL> DECLARE
    v_employee employees%ROWTYPE;
BEGIN
    INSERT INTO employees
    VALUES (9, 'Thida', 'IT', 700000, 'ACTIVE');

    UPDATE employees
    SET salary = salary + 150000
    WHERE department = 'IT';

    SELECT *
    INTO v_employee
    FROM employees
    WHERE employee_id = 9;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee Management Lab Completed');
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_employee.employee_name
);
    DBMS_OUTPUT.PUT_LINE('Updated Salary: ' || v_employee.salary);
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14   15
 16   17   18   19   20   21   22  Employee Management Lab Completed
Employee Name: Thida
Updated Salary: 850000

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   LAB 2: Customer Update Program
   Requirement:
   - Find customer
   - Update phone and city
   - Show updated result
   ============================================================ */
```

```
SQL> DECLARE
    v_customer customers%ROWTYPE;
BEGIN
    UPDATE customers
    SET phone = '099999999',
        city = 'Naypyidaw'
    WHERE customer_id = 1;

    SELECT *
    INTO v_customer
    FROM customers
    WHERE customer_id = 1;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Customer updated successfully');
    DBMS_OUTPUT.PUT_LINE('Customer Name: ' || v_customer.customer_name
);
    DBMS_OUTPUT.PUT_LINE('Phone: ' || v_customer.phone);
    DBMS_OUTPUT.PUT_LINE('City: ' || v_customer.city);
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14   15
 16   17   18   19   20   21  Customer updated successfully
Customer Name: Myanmar Telecom Co., Ltd
Phone: 099999999
City: Naypyidaw

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   LAB 3: Transaction Rollback Example
   Requirement:
   - Insert two employees
   - Save after first insert
   - Insert second employee
   - Rollback only second employee
   ============================================================ */
```

```
SQL> BEGIN
    INSERT INTO employees
    VALUES (10, 'Rollback Test 1', 'Training', 350000, 'ACTIVE');

    SAVEPOINT sp_employee_10;

    INSERT INTO employees
    VALUES (11, 'Rollback Test 2', 'Training', 360000, 'ACTIVE');

    ROLLBACK TO sp_employee_10;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Rollback Lab Completed');
    DBMS_OUTPUT.PUT_LINE('Employee 10 saved');
    DBMS_OUTPUT.PUT_LINE('Employee 11 rollbacked');
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14   15
 16   17   18  Rollback Lab Completed
Employee 10 saved
Employee 11 rollbacked

PL/SQL procedure successfully completed.
```

```
/* ============================================================
   FINAL CHECKING
   ============================================================ */
```

```
SQL> SELECT * FROM employees ORDER BY employee_id;

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                            SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         1
Aung Aung
IT                                                    950000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                            SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         2
Mg Mg
HR                                                    600000
ACTIVE

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                            SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         3
Su Su
Finance                                               900000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                            SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         5
Kyaw Kyaw
Admin                                                 450000
ACTIVE
```

```
EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                          SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         7
Savepoint User 1
IT                                                  550000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                          SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         9
Thida
IT                                                  850000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------------
DEPARTMENT                                          SALARY
-------------------------------------------------- ----------
STATUS
--------------------
        10
Rollback Test 1
Training                                            350000
ACTIVE
```

```
SQL> SELECT * FROM customers ORDER BY customer_id;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------
PHONE
------------------------------
CITY                                               STATUS
-------------------------------------------------- --------
         1
Myanmar Telecom Co., Ltd
099999999
Naypyidaw                                          ACTIVE


CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------
PHONE
------------------------------
CITY                                               STATUS
-------------------------------------------------- --------
         2
Global Trading Co., Ltd
092222222
Mandalay                                           ACTIVE
```

```
SQL> SELECT * FROM employee_backup ORDER BY employee_id;

EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------
DEPARTMENT                                          SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         1
Aung Aung
IT                                                  800000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------
DEPARTMENT                                          SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         2
Mg Mg
HR                                                  600000
ACTIVE


EMPLOYEE_ID
-----------
EMPLOYEE_NAME
--------------------------------------------------------------
DEPARTMENT                                          SALARY
-------------------------------------------------- ----------
STATUS
--------------------
         3
Su Su
Finance                                             900000
ACTIVE
```